



INSTITUTO DE CIÊNCIAS
TECNOLÓGICAS

<Local Database>

[UNIFEI | Universidade Federal de Itajubá](#)

Instituto de **C**iências **T**ecnológicas

ECO – <Dev Mobile>

eduardo.felipe@unifei.edu.br

Mongo Atlas DB

Antigo Realm DB

<https://www.mongodb.com/pt-br/docs/atlas/device-sdks/sdk/react-native/>

Características

- Desenvolvido para Mobile
- Performance
- Off line
- ~~Sincronismo automático~~ (descontinuado)
- NoSQL (schema)



Instalação

- No projeto React Native, proceder com a instalação da biblioteca "Realm (Atlas)"
 - `npm install realm`
 - `npm install @realm/react`
- MacOS
 - `cd ios && pod install && cd ..`
- <https://www.npmjs.com/package/realm/v/community>
- <https://www.mongodb.com/pt-br/docs/atlas/device-sdks/sdk/react-native/>



Arquitetura



- Schemas se assemelham a tabelas em bancos de dados relacionais (SQL).
- Crie uma pasta `src/models`
- Crie um arquivo denominado `Pessoas.jsx`
- Coloque o seguinte código:

```
const PessoaSchema = {  
  name: "Pessoa",  
  properties: {  
    _id: "int",  
    pNome: "string",  
    pIdade: "int",  
    pSexo: "string",  
    pTelefone: "string?",  
  },  
  primaryKey: "_id",  
}  
  
export default PessoaSchema;
```



- Todas as ações (leitura, criação, edição, exclusão, etc..) precisam de "saber qual banco" devem atuar.
- Crie uma pasta `src/infra`
- Crie um arquivo denominado `Banco.jsx`
- Coloque o seguinte código:

```
import Realm from "realm";  
import PessoaSchema from  
"../models/Pessoas";
```

```
const getBanco = async () => await  
Realm.open({  
  path: "pessoas",  
  schema: [PessoaSchema],  
})
```

```
export default getBanco;
```



Métodos para manipulação dos dados

- Crie uma pasta `src/services`
- Crie um arquivo `Dados.jsx`
- Neste arquivo vamos definir e testar alguns métodos para gravar dados e ler estes dados.



Gravar dados

```
import getBanco from "../infra/Banco"

const novoRegistro = async () => {
  console.log("Chamou novoRegistro");
  const banco = await getBanco();
  try {
    banco.write(() => {
      banco.create("Pessoa", {
        _id: 1,
        pNome: "Primeira pessoa",
        pIdade: 27,
        pSexo: "F",
        pTelefone: "31 988887777",
      })

      banco.create("Pessoa", {
        _id: 2,
        pNome: "Segunda pessoa",
        pIdade: 38,
        pSexo: "F",
        pTelefone: "31 988885555",
      })
    })
  } catch (erro) {
    console.log("Problema na criação de registro: ", erro);
  }
}

export default novoRegistro;
```



Ler dados

```
export const lerRegistros = async () => {  
  console.log("Chamou lerRegistros");  
  const banco = await getBanco();  
  try {  
    const informacoes = banco.objects("Pessoa");  
    console.log(informacoes.forEach( (item) => console.log(item)));  
  } catch (erro) {  
    console.log("Problema na leitura dos registros: ", erro);  
  }  
}  
  
export default novoRegistro;
```



Testando no App.tsx

```
import { Text, SafeAreaView, Button } from 'react-native'
import React from 'react'
import novoRegistro, {lerRegistros} from './src/services/Dados'

export default function App() {
  return (
    <SafeAreaView>
      <Text>App</Text>
      <Button
        title='criar'
        onPress={novoRegistro}/>
      <Button
        title='ler'
        onPress={lerRegistros}/>
    </SafeAreaView>
  )
}
```



Refatorando a função gravar dados

```
export const novoRegistro2 = async (obj) => {  
  console.log("Chamou novoRegistro2");  
  const banco = await getBanco();  
  try {  
    let registro;  
    banco.write(() => {  
      registro = banco.create("Pessoa", obj);  
      console.log(registro);  
    })  
    return registro;  
  } catch (erro) {  
    console.log("Problema na criação de registro: ", erro);  
  }  
}
```



```
import { Text, SafeAreaView, Button, TextInput } from 'react-native'

import React, { useState } from 'react'

import novoRegistro, {lerRegistros, novoRegistro2} from './src/services/Dados'

export default function App() {

  const [id, setId] = useState("");

  const [nome, setNome] = useState("");

  const [idade, setIdade] = useState("");

  const [sexo, setSexo] = useState("");

  async function gravarDados() {

    let dados = {};

    dados._id = parseInt(id);

    dados.pNome = nome;

    dados.pIdade = parseInt(idade);

    dados.pSexo = sexo;

    const resp = await novoRegistro2(dados);

    console.log(resp);

  }
```

```
return (

  <SafeAreaView>

    <Text>App</Text>

    <Button

      title='criar'

      onPress={novoRegistro} />

    <Button

      title='ler'

      onPress={lerRegistros} />

    <TextInpu t

      placeholder='identificador'

      value={id}

      onChangeText={setId} />

    <TextInpu t

      placeholder='nome'

      value={nome}

      onChangeText={setNome} />

    <TextInpu t

      placeholder='idade'

      value={idade}

      onChangeText={setIdade} />

    <TextInpu t

      placeholder='sexo'

      value={sexo}

      onChangeText={setSexo} />

    <Button

      title='Gravar dados'

      onPress={gravarDados}

    />

  </SafeAreaView>

)
```



Atualizar dados

```
export const atualizarRegistro = async (id, obj) => {  
  console.log("chamou atualizarRegistro");  
  const banco = await getBanco();  
  try {  
    let registro = banco.objects("Pessoa").filtered("_id = $0", id);  
    console.log(registro[0]);  
    console.log(obj);  
    banco.write( () => {  
      registro[0].pNome = obj.pNome;  
      registro[0].pIdade = obj.pIdade;  
      registro[0].pSexo = obj.pSexo;  
    });  
  } catch (erro) {  
    console.log("Problema na atualização do registro: ", erro);  
  }  
}
```



Refatorando a função atualizar dados

```
export const atualizarRegistro2 = async (obj) => {  
  console.log("chamou atualizarRegistro2");  
  const banco = await getBanco();  
  try {  
    banco.write( () => {  
      banco.create("Pessoa", obj, UpdateMode.Modified);  
    });  
  } catch (erro) {  
    console.log("Problema na atualização do registro: ", erro);  
  }  
}
```



Excluir dados

```
export const excluirRegistro = async (id) => {  
  console.log("chamou excluirRegistro", id);  
  const banco = await getBanco();  
  try {  
    banco.write( () => banco.delete(banco.objects("Pessoa").filtered("_id = $0", id)));  
  } catch (erro) {  
    console.log("Problema na exclusão do registro: ", erro);  
  }  
}
```



Chamar a função excluir dados

```
<Button  
  title='Excluir dados'  
  onPress={() => excluirRegistro(id)}  
/>
```



- <https://www.mongodb.com/pt-br/docs/atlas/device-sdks/>
- <https://youtube.com/playlist?list=PLT2gdUfk6jQTUAiwXTWai4rDo7pWX7by&si=yykXUahsNQ6PDbKI>





•

eduardo.felipe@unifei.edu.br